De tu primera landing a tu primer despliegue

MANUAL PRÁCTICO · VERSIÓN 1.0

React. Git. CI/CD.

Tu recorrido con Plotiart, explicado para repetirlo, entender los errores y continuar aprendiendo.

Proyecto: Plotiart · Diseño y publicidad

Responsable: Pedro Clemente

Fecha: 9 de noviembre de 2026 · Ecuador

Stack: React + TypeScript + Vite + Oxlint + GitHub Actions

> Este manual respalda el proceso y la configuración. No sustituye una copia del repositorio. Incluye el estado final y distingue los experimentos descartados de lo que quedó funcionando.

Cómo usar este manual

Sigue las páginas en orden si quieres reconstruir el aprendizaje. Para una tarea puntual, consulta el índice. Los recuadros distinguen comandos, contenido de archivos y resultados esperados.

Páginas	Contenido
3–5	Estructura del proyecto, React, menú, logo y pie de página
6–7	Git, repositorio local, ramas y primera subida
8–10	Comprobaciones, archivo de CI y Pull Requests
11–13	Vite, archivo de CD y publicación en Pages
14	Rutina para trabajar y publicar nuevas versiones
15–16	Errores reales y cómo diagnosticarlos
17	VPS, privacidad y protección de algoritmos
18	Respaldo, ejercicios y documentación oficial

TERMINAL · comando que puedes ejecutar

```
git status
```

ARCHIVO · fragmento para editar en vite.config.ts

```
base: '/plotiart/',
```

SALIDA DE EJEMPLO · no la ejecutes

```
3 files changed
```

> Copia únicamente bloques marcados TERMINAL. No copies el prompt, mensajes de salida, tablas ni fragmentos de TypeScript o YAML a la terminal. Una URL se abre en el navegador.

01 · Conoce tu proyecto

React construye la interfaz mediante componentes. TypeScript revisa el código. Vite ejecuta el entorno de desarrollo y prepara la versión publicable. Oxlint revisa reglas de código.

ESTRUCTURA · guía visual, no son comandos

```
plotiart/
├── .github/workflows/
│   ├── ci.yml
│   └── deploy.yaml
├── src/
│   ├── assets/plotiart-logo.png
│   ├── components/
│   │   ├── Menu/          ├── Inicio/
│   │   ├── Servicios/     ├── Portafolio/
│   │   ├── Nosotros/      ├── Contactos/
│   │   └── Piepagina/
│   ├── App.tsx
│   ├── App.css
│   ├── main.tsx
│   └── index.css
├── public/
├── package.json
├── package-lock.json
└── vite.config.ts
```

Archivo o carpeta	Responsabilidad
main.tsx	Monta React, activa StrictMode e importa el CSS global.
App.tsx	Reúne el menú, las secciones dentro de main y el footer.
components/	Cada sección vive en su propia carpeta. Inicio no contiene las demás secciones.
package.json	Scripts y dependencias. private: true evita publicar el paquete en npm; no hace privado GitHub.
package-lock.json	Versiones resueltas de dependencias; se conserva en Git.
node_modules/ y dist/	Dependencias instaladas y salida compilada; se regeneran.

02 · Componentes y estilos

Al revisar la plantilla, la compilación fallaba por variables sin uso: count, setCount y una importación de StrictMode. Se eliminaron los estados sobrantes y después se activó StrictMode envolviendo App. StrictMode ayuda a detectar problemas durante el desarrollo.

ARCHIVO · src/main.tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.tsx'
import './index.css'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

La página quedó organizada en Menu, Inicio, Servicios, Portafolio, Nosotros, Contacto y Piepagina. App conserva un único elemento main para el contenido principal; menú y pie quedan fuera.

El espacio exterior inicial venía del margen que el navegador aplica al body. El CSS que lo eliminaba no estaba importado. Se conectó index.css y se dejó esta base:

ARCHIVO · src/index.css, fragmento

```
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; }
```

> margin es espacio exterior; padding es espacio interior. Quitamos el margen del body, pero conservamos padding dentro del menú para que logo y enlaces no queden pegados al borde.

03 · Menú, navegación y marca

El menú usa useState para saber si está abierto en celular y useRef para devolver el foco al botón al presionar Escape desde los enlaces. El estado cambia la interfaz; la referencia permite acceder a un elemento DOM concreto.

CÓDIGO DE CONSULTA · fragmentos, no es un archivo completo

```
const [isOpen, setIsOpen] = useState(false)
const toggleRef = useRef<HTMLButtonElement>(null)

// Dentro del botón:
onClick={() => setIsOpen((open) => !open)}

// Al cerrar con Escape desde los enlaces:
setIsOpen(false)
toggleRef.current?.focus()
```

Se añadieron aria-expanded, aria-controls, nombres accesibles y estilos de foco. El menú adapta su distribución al ancho de pantalla y cierra al elegir un enlace.

Un enlace a /servicios solicita otra dirección al navegador. Como esta landing usa secciones, cambiamos las rutas por fragmentos:

ARCHIVO TSX · ejemplo de enlace y destino

```
<a href="#servicios">Servicios</a>
<section id="servicios">...</section>
```

El identificador debe coincidir exactamente. El desplazamiento suave está en App.css y se desactiva si el dispositivo pide reducir movimiento.

Logo: se probaron texto, una imagen generada, animación por recortes y un SVG geométrico. Los intentos de la letra A no gustaron y se volvió a la imagen PNG estática. No se afirma que la imagen generada sea idéntica al original. Logo.tsx conserva un experimento que el menú ya no importa.

Pie de página: Piepagina.tsx y Piepagina.css incluyen logo, llamada a contacto, navegación, servicios, año y enlace para volver al inicio. No se inventaron teléfonos ni correos.

04 · Git: local y remoto

Git registra versiones. GitHub aloja una copia remota. Tener una carpeta en tu equipo o crear un repositorio en GitHub no conecta ambos automáticamente.

Concepto	Qué significa
Repositorio local	Archivos e historial gestionados por la carpeta .git.
Rama	Una línea de trabajo dentro del mismo repositorio.
Working tree	Tus archivos tal como están en el disco.
Staging / índice	Contenido preparado para el próximo commit.
Commit	Una versión guardada localmente.
origin	Nombre asignado al repositorio remoto.
Push / fetch / pull	Enviar commits / descargar historial / descargar e integrar.
Pull Request	Propuesta de integrar una rama en otra; todavía no es el merge.

TERMINAL · ejecuta estos comandos

```
git status
git branch --show-current
git remote -v
git --no-pager log --oneline -5
```

Crear una rama desde src también afecta a todo el repositorio. Git busca la carpeta .git en los directorios superiores.

TERMINAL · ejecuta estos comandos

```
git switch -c devops/ci-cd
```

switch cambia de rama y -c crea una nueva desde tu posición actual. Usamos devops/ci-cd para CI y luego devops/deploy para el despliegue.

> El nombre de una rama organiza tu trabajo; no decide por sí solo cuándo corre CI. Eso se configura con los eventos del workflow.

05 · Primera subida y conexión local

El repositorio se creó en PedroClemente150198/plotiart. La primera subida se hizo desde una copia temporal porque la sesión tenía restringida .git. Por eso al principio git status fallaba en la carpeta original. Después se inicializó Git allí, se descargó el historial y se vinculó main con origin/main, conservando los archivos.

El remoto final utiliza SSH:

DIRECCIÓN REMOTA · no es un comando

```
git@github.com:PedroClemente150198/plotiart.git
```

En un proyecto nuevo que todavía no tiene historial ni remoto, una secuencia inicial sería:

TERMINAL · ejecuta estos comandos

```
git init -b main
git add .
git commit -m "Versión inicial del proyecto"
git remote add origin git@github.com:PedroClemente150198/plotiart.git
git push -u origin main
```

> No repitas esa inicialización en tu proyecto actual: ya está conectado. Si el remoto ya tiene commits, clónalo para empezar desde su historial en lugar de crear otro historial independiente.

El .gitignore excluye node_modules, dist, variables de entorno, configuraciones locales y output. Se subieron src, public, configuración y package-lock.json. Un archivo ya registrado no deja de estarlo por añadirlo después al .gitignore.

La clave privada SSH permanece en tu equipo; no se copia al repositorio. Si HTTPS pide credenciales y SSH ya funciona, el remoto SSH permite reutilizar ese acceso.

Primer commit: **0fce4e8**, “Versión inicial de la landing de Plotiart”. Esta referencia documenta el punto inicial, no debe copiarse para sobrescribir versiones posteriores.

06 · Validar antes de subir

Antes de automatizar, ejecutaste manualmente los mismos pasos que luego haría GitHub. Trabaja desde la raíz, donde está package.json.

TERMINAL · ejecuta estos comandos

```
npm ci
npm run lint -- --deny-warnings
npm run build
```

Comando	Qué comprueba o produce
npm ci	Instala dependencias conforme al lockfile. Sustituye node_modules y falla si package.json y el lockfile no concuerdan.
npm run lint -- --deny-warnings	Ejecuta Oxlint. El separador -- pasa la opción al linter para tratar advertencias como fallo.
npm run build	Ejecuta tsc -b y vite build. Revisa la compilación y genera dist.
npm run dev	Servidor de desarrollo con actualización al editar.
npm run preview	Vista local de la compilación; no es un servidor de producción.

Tus resultados fueron: instalación completada, 0 advertencias, 0 errores y compilación exitosa. El aviso de financiación de npm era informativo. PLUGIN_TIMINGS informaba tiempos de plugins; no significaba que la compilación había fallado.

Puedes comprobar visualmente la compilación:

TERMINAL · ejecuta estos comandos

```
npm run preview
```

Abre la URL que muestre Vite incluyendo /plotiart/. Detén el servidor con Ctrl+C.

> Lint y build no sustituyen las pruebas de comportamiento. Este proyecto todavía no tiene una suite automatizada que compruebe, por ejemplo, abrir el menú o navegar por teclado.

07 · CI: archivo de referencia

Ubicación real: **.github/workflows/ci.yml**. Este contenido corresponde al archivo revisado al generar el manual. YAML usa espacios para la sangría.

ARCHIVO YAML · .github/workflows/ci.yml

```yaml
name: CI PLOTIART

on:
  pull_request:
    branches: [main]
  workflow_call:
    inputs:
      publish:
        description: Preparar el artefacto para GitHub Pages
        type: boolean
        default: false

permissions:
  contents: read

jobs:
  validate:
    name: Revisar y compilar
    runs-on: ubuntu-latest

    steps:
      - name: Descargar código
        uses: actions/checkout@v6

      - name: Preparar Node
        uses: actions/setup-node@v6
        with:
          node-version: 24
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Revisar código
        run: npm run lint -- --deny-warnings

      - name: Compilar proyecto
        run: npm run build

      - name: Guardar sitio compilado
        if: inputs.publish == true
        uses: actions/upload-pages-artifact@v4
        with:
          path: dist
```

08 · CI y tu primer Pull Request

Bloque de CI	Cómo funciona
pull_request → branches: [main]	Valida propuestas cuyo destino es main, al abrirlas o actualizarlas.
workflow_call	Permite que otro workflow reutilice este trabajo.
inputs.publish	Booleano; false por defecto. CD lo activa para guardar el resultado.
contents: read	Permiso de lectura del código para el token del workflow.
validate / ubuntu-latest	Identificador del trabajo y entorno de ejecución Linux.
checkout / setup-node	Obtiene el código y prepara Node 24 con caché de npm.
upload-pages-artifact	Empaqueta dist para que el trabajo de publicación lo utilice. Solo se ejecuta si publish es true.

Al abrir un Pull Request no publicamos nada. El trabajo instala, revisa y compila. En la ejecución de CD, ese mismo workflow también guarda el artefacto.

TERMINAL · ejecuta estos comandos

```
git add .github/workflows/ci.yml
git --no-pager diff --cached
git commit -m "Configura CI para los pull requests"
git push -u origin devops/ci-cd
```

En el navegador: repositorio → Pull requests → New pull request. Elige base: main y compare: devops/ci-cd. Revisa los cambios, pon un título y crea la propuesta.

En Checks o Actions abre “Revisar y compilar”. Amarillo significa pendiente o en ejecución; verde, éxito; rojo, fallo. En tu primer PR la comprobación terminó correctamente. Después integraste con Merge pull request → Confirm merge.

> CI verde no impide por sí solo integrar código que falla en otra ocasión. Para exigir estas comprobaciones hay que configurar reglas de protección o un ruleset. Esa protección no se configuró durante este recorrido.

TERMINAL · ejecuta estos comandos

```
git switch main
git pull --ff-only origin main
```

09 · Preparar la publicación

GitHub Pages publica archivos estáticos. Vite prepara esos archivos en dist; no publicamos el servidor de desarrollo ni node_modules.

Para el repositorio plotiart se configuró esta ruta base:

ARCHIVO · vite.config.ts

```
import react, { reactCompilerPreset } from '@vitejs/plugin-react'
import babel from '@rolldown/plugin-babel'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  base: '/plotiart/',
  plugins: [
    react(),
    babel({ presets: [reactCompilerPreset()] })
  ],
})
```

El sitio se publica bajo /plotiart/. base permite construir las direcciones correctas de los recursos. Si en el futuro publicas en la raíz de un dominio propio, normalmente usarías / como base.

Durante la implementación se creó la rama devops/deploy. Se actualizaron CI y Vite, y se añadió deploy.yaml. Las extensiones .yml y .yaml funcionan, pero Git requiere que uses el nombre real del archivo.

TERMINAL · ejecuta estos comandos

```
git add .github/workflows/ci.yml .github/workflows/deploy.yaml vite.config.ts
git --no-pager diff --cached --stat
git commit -m "Configura despliegue manual a GitHub Pages"
git push -u origin devops/deploy
```

Se abrió un segundo Pull Request hacia main. Una vez CI pasó, se integró. El workflow manual debe existir en la rama principal para aparecer como ejecución manual en Actions.

> Tu cambio de despliegue quedó registrado en el commit 653079d. Los identificadores de commit sirven para localizar y comparar versiones; no son comandos.

10 · CD: archivo de referencia

Ubicación real: **.github/workflows/deploy.yaml**. El despliegue sigue siendo manual: un push o un merge por sí solos no publican la página.

ARCHIVO YAML · .github/workflows/deploy.yaml

```yaml
name: CD PLOTIART

on:
    workflow_dispatch:

permissions:
    contents: read

concurrency:
    group: github-pages
    cancel-in-progress: false

jobs:
    build:
        name: Validar y preparar sitio
        if: github.ref == 'refs/heads/main'
        uses: ./.github/workflows/ci.yml
        with:
            publish: true

    deploy:
        name: Publicar en GitHub Pages
        needs: build
        runs-on: ubuntu-latest

        permissions:
            pages: write
            id-token: write

        environment:
            name: github-pages
            url: ${{ steps.deployment.outputs.page_url }}

        steps:
            - name: Configurar GitHub Pages
              uses: actions/configure-pages@v5

            - name: Publicar sitio
              id: deployment
              uses: actions/deploy-pages@v4
```

11 · Entender y ejecutar CD

Bloque de CD	Responsabilidad
workflow_dispatch	Habilita el botón Run workflow.
concurrency	Limita publicaciones simultáneas del mismo grupo; no garantiza desplegar toda ejecución intermedia.
if: github.ref == refs/heads/main	Permite construir para publicación únicamente desde main.
uses: ./.github/workflows/ci.yml	Llama al CI local del mismo commit. No se duplican los comandos.
publish: true	Activa la subida del artefacto de Pages.
needs: build	Espera a que la preparación termine correctamente.
pages: write / id-token: write	Permisos que requiere el trabajo de publicación.
environment: github-pages	Registra el destino del despliegue y su enlace. No equivale por sí solo a exigir aprobación humana.
deploy-pages	Publica el artefacto producido por el trabajo anterior.

Configuración que hiciste en el navegador:

- El repositorio inicialmente era privado. Pages mostró que con ese plan había que cambiar visibilidad o actualizar el plan. Decidiste hacerlo público.
- En Settings → Pages → Build and deployment → Source seleccionaste GitHub Actions. No había que configurar las tarjetas de Jekyll o Static HTML.
- El campo Custom domain quedó vacío; utilizamos la dirección de GitHub Pages.
- En Actions elegiste CD PLOTIART → Run workflow → main → Run workflow.

El trabajo “Validar y preparar sitio” terminó con éxito y guardó el sitio compilado. Después “Publicar en GitHub Pages” también terminó con éxito. Esa fue la evidencia del primer despliegue.

Web: pedroclemente150198.github.io/plotiart/
Ejecución: CD PLOTIART · primera publicación

12 · Tu rutina para el próximo cambio

Este es el ciclo que puedes repetir. El nombre de rama y el mensaje son ejemplos: elige otros que describan tu siguiente tarea.

TERMINAL · ejecuta estos comandos

```
git switch main
git pull --ff-only origin main
git switch -c mejora/texto-inicio
```

1. Confirma que no tienes cambios pendientes antes de cambiar de tarea. Edita el componente correspondiente y guarda el archivo. Comprueba la interfaz con npm run dev.

TERMINAL · ejecuta estos comandos

```
npm run lint -- --deny-warnings
npm run build
git status
git diff
```

2. Cuando esté correcto, prepara únicamente los archivos de esa tarea. Este ejemplo asume que editaste Inicio.tsx:

TERMINAL · ejecuta estos comandos

```
git add src/components/Inicio/Inicio.tsx
git --no-pager diff --cached
git commit -m "Mejora el texto de inicio"
git push -u origin mejora/texto-inicio
```

3. En GitHub abre el PR hacia main, revisa Files changed y espera CI verde. Integra la propuesta. Después ejecuta CD PLOTIART manualmente desde main si quieres publicar ese cambio.

4. Revisa que la ejecución termine correctamente y prueba el sitio publicado: logo, estilos, enlaces y menú en móvil. Una publicación exitosa no reemplaza la comprobación visual.

TERMINAL · ejecuta estos comandos

```
git switch main
git pull --ff-only origin main
```

> Cada publicación debe poder vincularse con un commit. Si necesitas recuperar una versión, identifica el cambio, crea una corrección o un revert mediante PR y publica de nuevo. No necesitas sobrescribir el historial con force push.

13 · Errores de terminal que resolvimos

Mensaje o síntoma	Causa y solución
fatal: no es un repositorio git	La carpeta no tenía metadatos válidos de Git. Se inicializó y conectó al historial existente. Si estás en otra carpeta, revisa primero pwd y git status.
Ruta deploy.yml no coincide con ningún archivo	El archivo real se llamaba deploy.yaml. Revisa ls -la .github/workflows y utiliza el nombre exacto.
zsh: command not found: base:	Se pegó TypeScript en la terminal. base va dentro de defineConfig en vite.config.ts.
Permiso denegado al escribir ci.yml como comando	Se intentó ejecutar un archivo de configuración. Ábrelo en el editor; no cambies sus permisos para arreglar esto.
command not found: 14 o 40	Se pegó el resumen de un diff como si fueran comandos. Las cifras y barras del resumen son salida informativa.
La terminal muestra una ✗	El tema del prompt puede marcar archivos modificados o sin seguimiento. git status es la referencia para entender el estado real.
Everything up-to-date	No había commits nuevos que enviar a esa rama remota. No es un fallo.
git diff parece no mostrar nada	Puede no haber diferencias o haberse abierto un visor. Usa git --no-pager diff --cached --stat y git status. Para salir del visor, q.

Un error al ejecutar un texto equivocado no significa automáticamente que hayas perdido el código. Primero consulta el estado y lee el mensaje completo; no recurras a reset --hard ni a comandos de borrado para intentar arreglarlo.

TERMINAL · ejecuta estos comandos

```
pwd
git status
ls -la .github/workflows
git --no-pager diff --cached --stat
```

14 · Errores de interfaz y publicación

Problema	Qué revisar
Espacio alrededor del menú	Importación del CSS global y body { margin: 0; }.
La página se recarga al pulsar un enlace	Para secciones usa #inicio y #servicios, con id coincidente. Para varias vistas reales se necesitaría otra solución de navegación.
No encuentra el logo tras mover Menu.tsx	Desde components/Menu/Menu.tsx la imagen está en ../../assets/plotiart-logo.png. Cada ../ sube un nivel.
El botón de contacto aparece duplicado	Si ya está en el array links, genéralo una sola vez en map y aplica la clase según isContact.
La A del logo se deforma	El recorte de una imagen no separa de manera limpia una letra. El estado final usa PNG sin animación; el SVG experimental no está conectado.
No aparece Run workflow	Comprueba que el YAML incluya workflow_dispatch y esté integrado en la rama principal.
Build o deploy en rojo	Abre la ejecución en Actions, entra al trabajo y lee el primer paso fallido. No vuelvas a ejecutar sin entender qué falló.
La web no carga recursos	Comprueba base: /plotiart/, el nombre exacto del repositorio y las direcciones de recursos en el navegador.
404 después de publicar	Comprueba la URL completa y el estado del despliegue. Un 404 de github.com puede ser un enlace al repositorio equivocado; no demuestra un fallo de la web en github.io.

Durante el proceso apareció una captura de 404 sin dirección visible. Se consultó Actions y ambos trabajos ya habían terminado correctamente. La dirección correcta para visitar la web es la de github.io, incluyendo /plotiart/.

> El portafolio y el contacto tienen textos provisionales. Publicar la landing no crea un formulario, no envía correos ni añade canales de contacto reales. Esas funcionalidades quedan para próximos cambios.

15 · VPS y algoritmos privados

Una VPS es un servidor que administras. No se configuró ninguna durante este recorrido; esta página resume la arquitectura que hablamos como siguiente etapa.

Capa	Qué haría
CI en GitHub Actions	Valida y compila; genera dist.
CD para VPS	Envía el artefacto mediante SSH y activa una nueva versión.
Nginx	Entrega los archivos estáticos al navegador.
Dominio y HTTPS	Apuntan al servidor y protegen el tráfico.
Operación	Actualizaciones, accesos, monitoreo, respaldo y recuperación a tu cargo.

Para esta landing estática no hace falta mantener Node ejecutándose en producción. Tampoco se usa npm run dev o npm run preview como servidor público. Si publicas en la raíz de un dominio, adapta la base de Vite.

Qué significa proteger tus algoritmos: el navegador recibe el HTML, CSS y JavaScript del frontend, incluso con un repositorio privado o una VPS. Minificar u ofuscar no lo convierte en secreto.

ARQUITECTURA · ejemplo conceptual

```
Navegador (React)
      │ envía datos por HTTPS
      ▼
API en el servidor
      │ ejecuta la lógica privada
      ▼
Respuesta con el resultado, sin entregar el código fuente
```

Una lógica sensible debe ejecutarse en el backend, con autenticación y límites adecuados. No envíes claves privadas al frontend. Un repositorio privado protege el acceso al repositorio; no evita que los visitantes inspeccionen el código que reciben.

> Una VPS tampoco impide por sí sola el acceso del proveedor de infraestructura al servidor. Si ese fuera tu requisito, habría que estudiar otro modelo de confianza. No se implementó ningún backend ni migración a VPS.

16 · Respaldo y próximos ejercicios

Guarda dos cosas: este manual y una copia del código con su historial. El PDF documenta el aprendizaje; el repositorio contiene los archivos que puedes reconstruir.

TERMINAL · recuperar el proyecto en una carpeta nueva

```
git clone git@github.com:PedroClemente150198/plotiart.git
cd plotiart
npm ci
npm run dev
```

Ejecuta el clon desde una carpeta padre donde aún no exista plotiart. Para crear además un respaldo transportable de los commits y referencias locales:

TERMINAL · ejecuta estos comandos

```
git bundle create ../plotiart-respaldo.bundle --all
```

El bundle no incluye cambios sin commit, archivos sin seguimiento ni secretos ignorados. No guarda la configuración de Pages, los secrets ni el historial de ejecuciones de Actions. Guarda esas configuraciones por separado cuando sean necesarias.

Ejercicios para continuar:

- Añade una tarjeta de servicio en una rama nueva y recorre el ciclo PR → CI → merge → CD.
- En una rama de práctica, deja una variable sin uso, observa el fallo y corrígelo. No integres el error.
- Añade una prueba útil del menú móvil; luego incorpora su ejecución a CI.
- Configura una regla que exija CI antes de integrar en main. Comprueba las condiciones de tu cuenta.
- Optimiza el PNG del logo conservando el original y compara el resultado visual.
- Más adelante, practica staging, versiones de acciones fijadas por SHA y despliegues recuperables.

Documentación oficial de consulta:

- React · aprender componentes y estado
- React · referencias con useRef
- Vite · compilación y despliegue estático
- GitHub · reutilizar workflows
- GitHub · workflows para Pages

Configuración transcrita del proyecto al 9 de noviembre de 2026. Las versiones de herramientas y opciones de GitHub pueden cambiar: revisa las fuentes al actualizar el pipeline.